LLVM

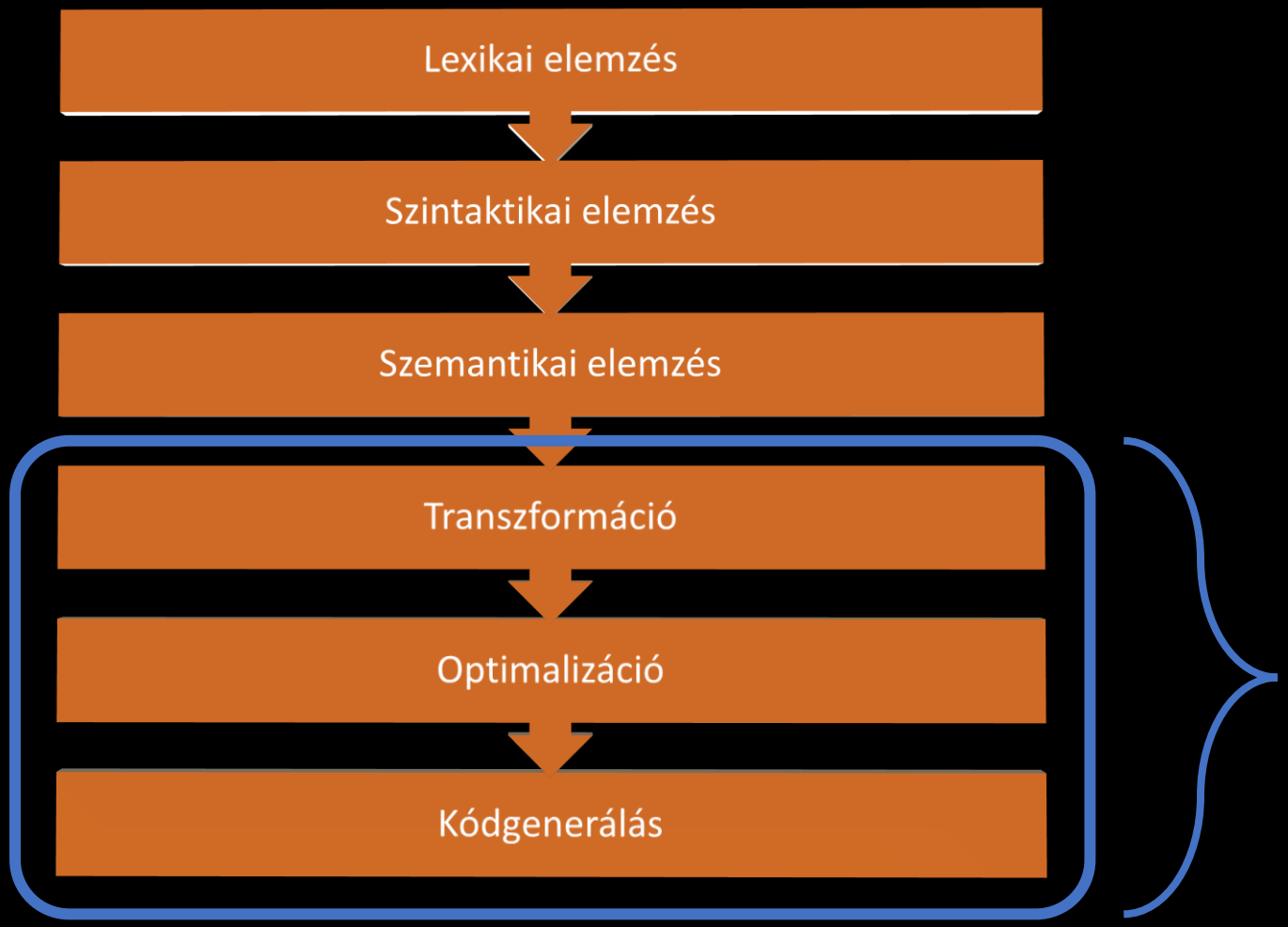
Gergely Gembela

Model-based Software Development

Mi is az LLVM?

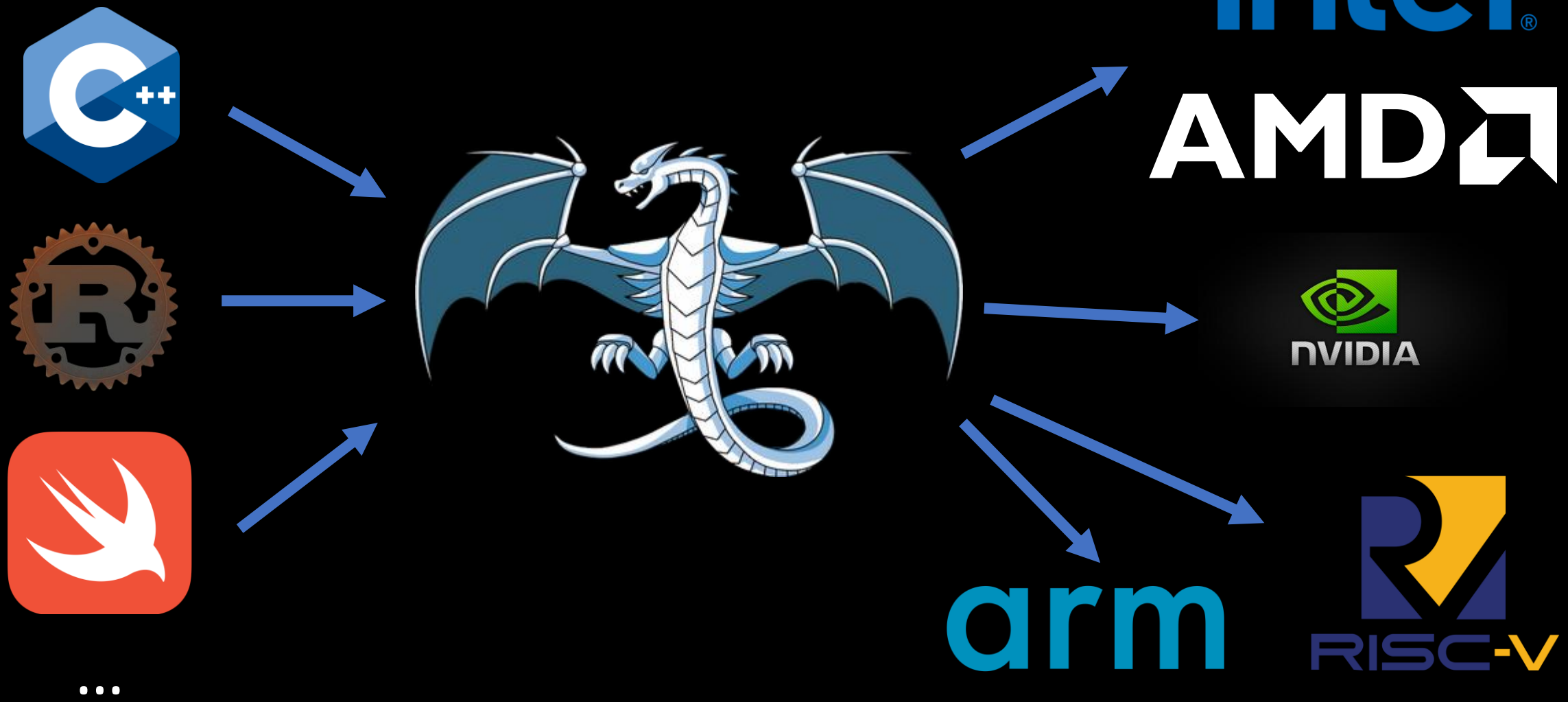
- Fordítók készítéséhez és teszteléséhez használható eszközök gyűjteménye (és már NEM a Low-Level Virtual Machine mozaikszava)
- Több népszerű nyelvhez van LLVM alapú fordító, vannak nyelvek amik kifejezetten LLVM alapú fordítóikkal együtt fejlődnek
  - Rust
  - Zig
  - Nim
  - Clang – C++, Objective C, C
  - Swift

Az LLVM feladatai egy fordítóban



Middle-end és backend: Az LLVM eszközei első sorban a fordítás ezen szakaszára adnak megoldást

Mi az alapvető cél?



A sárkányon belül

- IR nyelv
  - Frontendfüggetlen*
  - Platformfüggetlen*
- Optimalizáció
  - Kiterjeszthető, személyre szabható lépések
  - Az IR nyelven elérhetők
- Platformspecifikus backendek
- Linker
  - Képes link-time optimalizációkra

Egy modern LLVM alapú fordító

Lexer+Parser

AST

Egy modern LLVM alapú fordító

Lexer+Parser

AST

Semantic analysis

High-level IR

Language specific optimiaztions

Transformations

LLVM IR Generation

Egy modern LLVM alapú fordító

Lexer+Parser

AST

Semantic analysis

High-level IR

Language specific optimiaztions

Transformations

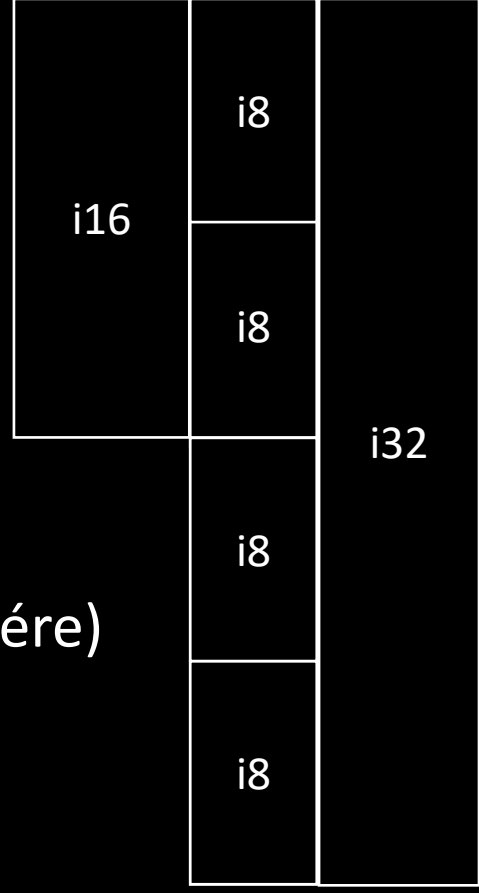
LLVM IR Generation

Optimizations

Target-specific backend

Link-time optimizations (lld)

LLVM IR



- 3 formában használjuk – bitcode, in-memory IR, szöveges
- Az LLVM IR típusos `%struct.vec4 = type { i16, i16, i16, i16 }`
  - Régebben még a pointerek is, ez mostanra eltűnőben van (szerencsére)
- Definiálhatók saját típusok
  - Ezek leginkább a C struktúrákhoz hasonlítanak, DE alignment!
- 2 féle ID – minden elemhez (változó, függvény, modul)
  - @global – globális
  - %local – lokális
- A kódot modulokba szervezi (egy fordítási egység, pl. egy .cpp file)
  - Jelenleg csak modulon belüli optimalizációkkal foglalkozunk

@square – C++

```c
int square(int num) {
    return num * num;
} //C/C++
```

```llvm
define i32 @square(int)(i32 noundef
%0) {
  %2 = mul nsw i32 %0, %0
  ret i32 %2
} ;LLVM IR
```

Compiler explorer

@square – rs

```rust
pub fn square(num: i32) -> i32 {
    num * num
} //RS
```

```llvm
define i32 @square(int)(i32 noundef
%0) {
  %2 = mul nsw i32 %0, %0
  ret i32 %2
} ;LLVM IR
```

Compiler explorer

@square – Swift

```swift
func square(n: Int) -> Int {
    return n * n
} //Swift
```

```llvm
define i32 @square(int)(i32 noundef
%0) {
  %2 = mul nsw i32 %0, %0
  ret i32 %2
} ;LLVM IR
```

Compiler explorer

@square – llc (debug/release/ssa/…)

```
define dso_local noundef i32
@square(int)(i32 noundef %0) #0 !dbg !10 {
  %2 = alloca i32, align 4
  store i32 %0, ptr %2, align 4
  tail call void @llvm.dbg.declare(metadata
ptr %2, metadata !16, metadata
!DIExpression()), !dbg !17
  %3 = load i32, ptr %2, align 4, !dbg !18
  %4 = load i32, ptr %2, align 4, !dbg !19
  %5 = mul nsw i32 %3, %4, !dbg !20
  ret i32 %5, !dbg !21
}
```

```
define i32 @square(int)(i32 noundef
%0) {
  %2 = mul nsw i32 %0, %0
  ret i32 %2
} ;LLVM IR
```

@square – IR to asm

```
square:
      # x86-64
        mov     eax, edi
        imul    eax, eax
        ret
```

```
square(int): # WASM
        local.get       0
        local.get       0
        i32.mul
        end_function
```

```
define i32 @square(int)(i32 noundef %0)
{
  %2 = mul nsw i32 %0, %0
  ret i32 %2
} ;LLVM IR
```

```
square:   //arm64
mul     w0, w0, w0
ret
```

```
square: # RISC-V
mul     a0, a0, a0
ret
```

nvcc

```
.visible .entry square(int*, int)(
        .param .u64 square(int*, int)_param_0,
        .param .u32 square(int*, int)_param_1
)
{

        ld.param.u64    %rd1, [square(int*, int)_param_0];
        ld.param.u32    %r1, [square(int*, int)_param_1];
        cvta.to.global.u64      %rd2, %rd1;
        mul.lo.s32      %r2, %r1, %r1;
        st.global.u32   [%rd2], %r2;
        ret;

}
```

LLVM Pass (IR->IR)

- Kód2kód transzformáció
- Akár mi is írhatunk
- De van *néhány* ami már meg van írva, és nyelvfüggetlenül elérhető
  - Érdekesség: ennyire egyszerű egy saját pass-t készíteni

```
#include "llvm/IR/PassManager.h"

namespace llvm {

class HelloWorldPass : public PassInfoMixin<HelloWorldPass> {
public:
  PreservedAnalyses run(Function &F, FunctionAnalysisManager &AM);
};

} // namespace llvm
```

Optimalizálás LLVM segítségével

- Az összes tanult optimalizáló algoritmus elérhető, és akár saját implementáció is készíthető rájuk
- Sok pass nem része az alapértelmezetten engedélyezett szinteknek (Itt a szintek Pl. O[0-3], Ofast, Os, Oz)
- Nem minden optimalizáció platformfüggetlen!
  - Pl. SIMD instrukciók generálása (SSE/AVX, Arm Neon,)
- Egy később futó pass visszavonhatja egy előtte futó transzformációját
- A passek külön is tesztelhetők az opt eszközzel, pl.
  - `opt -passes='loop-unroll'`
  - `opt -passes='dce'`

Az optimalizáló pipeline megtekintése

C++ source #1

C++

```cpp
#include <stdio.h>

inline long factorial(long n){
    long res = 1;
    for(long i = 2; i <= n; i++){
        res *= i;
    }
    return res;
}

int main() {
    volatile auto a = factorial(14);
}
```

armv8-a clang (trunk) (Editor #1)

armv8-a clang (trunk)

-O2 -emit-llvm

- Clone Compiler
- Optimization
- Stack Usage
- Preprocessor
- AST
- LLVM IR
- Opt Pipeline
- Device
- Control Flow Graph

```llvm
ef i32 @main() local_unnamed_addr #0 !dbg

gn 8
.dbg.assign(metadata i1 undef, metadata !
time.start.p0(i64 8, ptr nonnull %a), !db
7178291200, ptr %a, align 8, !dbg !23
.dbg.assign(metadata i64 poison, metadata
time.end.p0(i64 8, ptr nonnull %a), !dbg

etime.start.p0(i64 immarg, ptr nocapture)

etime.end.p0(i64 immarg, ptr nocapture) #

declare void @llvm.dbg.assign(metadata, metadata, metadata, met

attributes #0 = { mustprogress nofree norecurse nounwind memory
attributes #1 = { mustprogress nocallback nofree nosync nounwin
attributes #2 = { nocallback nofree nosync nounwind speculatabl

!20 = distinct !DIAssignID()
!28 = distinct !DIAssignID()
```

Az optimalizáló pipeline megtekintése

Opt Pipeline Viewer armv8-a clang (trunk) (Editor #1, Compiler #1)

Options Filters Function: main

Passes:

- AssignmentTrackingPass on [module]
- GlobalOptPass on [module]
- InstCombinePass on main
- InlinerPass on (main)
- SROAPass on main
- CorrelatedValuePropagationPass on main
- ReassociatePass on main
- LCSSAPass on main
- InstCombinePass on main
- LCSSAPass on main
- IndVarSimplifyPass on for.body.i
- LoopDeletionPass on for.body.i
- SROAPass on main
- GVNPass on main
- InstCombinePass on main
- ADCEPass on main
- SROAPass on main
- InstCombinePass on main

```
-define dso_local noundef i32 @main() {
1+define dso_local noundef i32 @main() local_unnamed_addr {
2 entry:
3   %a = alloca i64, align 8
4     #dbg_assign(i1 undef, !17, !DIExpression(), !20, ptr %a, !DIExpression(), !21)
5   call void @llvm.lifetime.start.p0(i64 8, ptr %a)
6   %call = call noundef i64 @factorial(long)(i64 noundef 14)
7   store volatile i64 %call, ptr %a, align 8
8     #dbg_assign(i64 %call, !17, !DIExpression(), !29, ptr %a, !DIExpression(), !21)
9   call void @llvm.lifetime.end.p0(i64 8, ptr %a)
10  ret i32 0
11 }
```

Example: dead code elimination

```
long dce(long n){
    long res = 1;
    auto c = res - 3;
    return res;
}
```

Compiler explorer

Example: Mem2Reg (SROA+Mem2Reg)

```
long sroa(long n){
    long res = 1;
    return res + n;
}
```

Compiler explorer

Example: deletion (IndVarSimplify, LoopDeletion, SimplifyCFG)

```
int whatever42(int a){

    int x = 3;
    for(int i = 0; i < a; i++){
        int c = a*a;
        x += c;
    }
    return x;
}
```

Compiler explorer

Example: constant inlining

```
inline long factorial(long n){
    long res = 1;
    for(long i = 2; i <= n; i++){
        res *= i;
    }
    return res;
}

int main() {
    volatile auto a = factorial(14);
}
```

Compiler explorer

Example: loop unroll

```
void vecadd(float a[4], float b[4]){
    for(int i = 0; i < 4; i++){
        a[i] = b[i];
    }
}
```

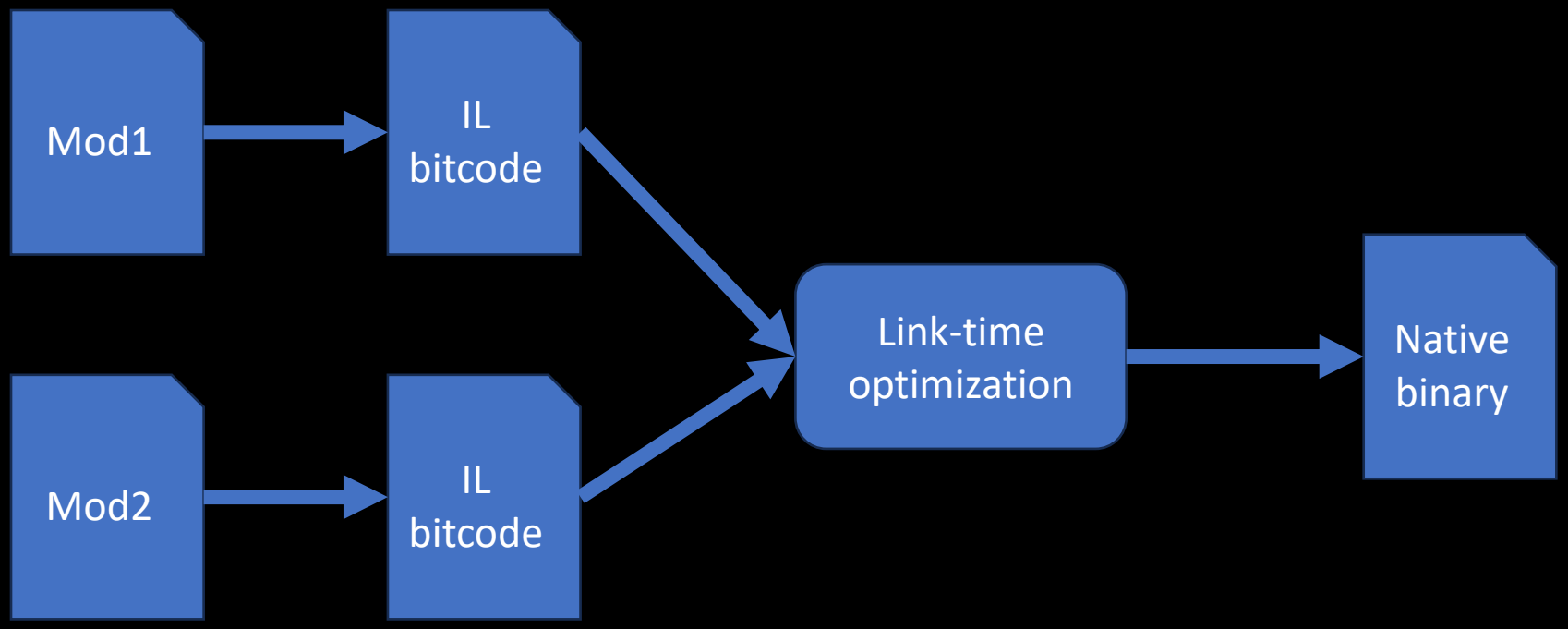
Compiler explorer

Extreme example: dead code elimination #2

```
fn square(num: i32) -> i32 {
    num * num
} //RS
```

Link-time optimization

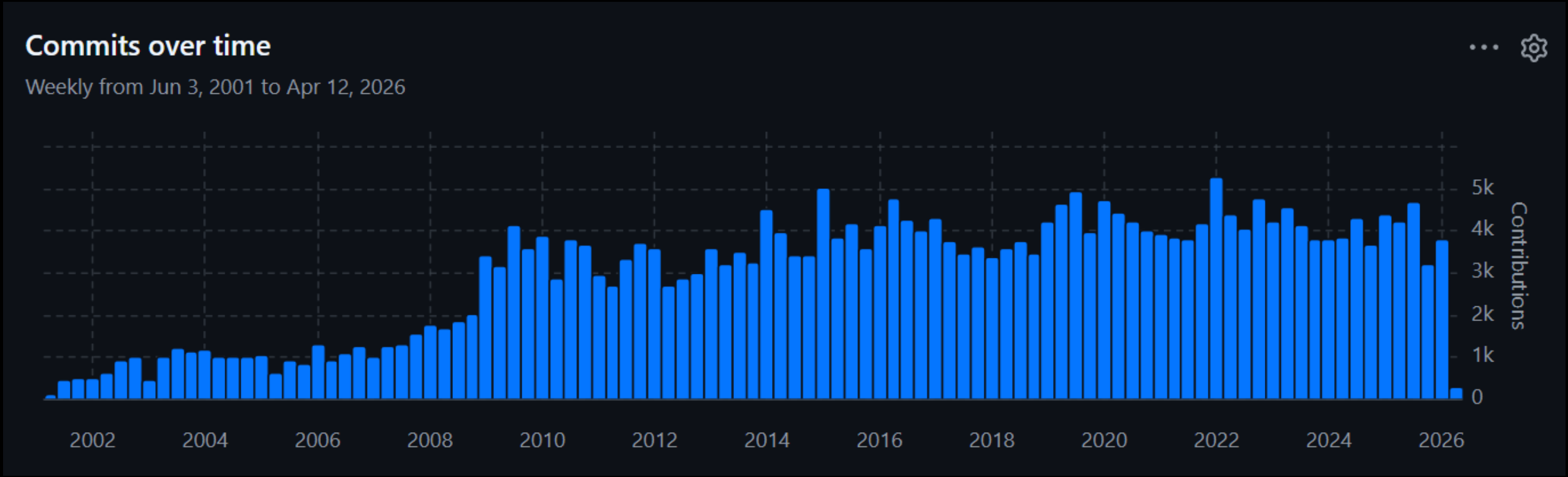
- An inter-module level optimization, capable of optimizing the whole program
- Increases compilation time, as parallelization is problematic in this stage



Companies that use LLVM

Stats

576,709 Commits



Köszönöm a figyelmet!

Sources, additional material

The Architecture of Open Source Applications (Volume 1)LLVM (aosabook.org)
LLVM IR and Go | Gopher Academy Blog
Using the New Pass Manager — LLVM 19.0.0git documentation
Compiling With Clang Optimization Flags – Incredibuild

LLVM IR:
LLVM Assembly Language Reference Manual (apple.com)

Videos:
LLVM in 100 Seconds (youtube.com)
(5) 2023 EuroLLVM - Tutorial: A whirlwind tour of the LLVM optimizer – YouTube